WEEK 7 – Testing Mastery

Welcome to Week 7 - Testing Mastery for Developers. This week focuses on writing robust test code using Jest, the industry-standard JavaScript testing framework. You'll learn to write unit tests, mock dependencies, test async code, and achieve comprehensive test coverage. By the end of this week, you'll be confident writing test suites that catch bugs before they reach production.

WEEKS 7-8

HANDS-ON MASTERY

Unit Testing Fundamentals with Jest

Unit testing is a cornerstone of modern software development, empowering developers to ensure the reliability and correctness of their code. For a developer, a unit test means writing small, isolated tests that verify the behavior of the smallest testable parts of an application—typically individual functions, methods, or components. The goal is to catch bugs early, simplify debugging, and provide confidence when refactoring or adding new features.

Jest is a popular JavaScript testing framework, widely adopted for its simplicity and powerful features. When writing tests with Jest, a common and highly effective pattern is **AAA: Arrange, Act, Assert**. This structure promotes clear, readable, and maintainable tests:

- **Arrange:** Set up the test environment, declare variables, and prepare any necessary data or mocks.
- **Act:** Execute the specific function or code under test.
- **Assert:** Verify that the outcome of the action is as expected, using Jest's matchers.

Jest test files typically follow naming conventions like `*.test.js` or `*.spec.js`, making them easily discoverable by the test runner. Tests are organized using `describe()` blocks for grouping related tests and `test()` (or `it()`) for individual test cases.

Example: Calculator Functions and Jest Tests

Consider a simple `calculator.js` module:

```
// calculator.js
function add(a, b) {
  return a + b;
}

function divide(a, b) {
  if (b === 0) throw new Error('Cannot divide by zero');
  return a / b;
}

module.exports = { add, divide };
```

Here's how you'd write Jest tests for it, demonstrating the AAA pattern:

```
// calculator.test.js
const { add, divide } = require('./calculator');

describe('Calculator Functions', () => {
  test('adds two numbers correctly', () => {
    // Arrange
    const num1 = 5;
    const num2 = 3;

    // Act
    const result = add(num1, num2);

    // Assert
    expect(result).toBe(8);
  });

  test('throws error when dividing by zero', () => {
    // Arrange & Act (implicit within expect(() => ...).toThrow())
    // Assert
    expect(() => divide(10, 0)).toThrow('Cannot divide by zero');
  });
});
```

Exercise Activity

Task: Write a complete test suite for a `validateUser` function that checks: email format (valid/invalid), password strength (min 8 chars, has number, has special char), and username length (3-20 chars). Include at least 8 test cases covering happy path and edge cases.

Expected Output / Artifact: A `.test.js` file with `describe` blocks, 8+ test cases using Jest matchers (`toBe`, `toMatch`, `toThrow`, `toBeTruthy`), and all tests passing when run with `npm test`.

Assessment Type: Code submission + automated test execution

Assessment Criteria:

- Test coverage of all validation rules (30%)
- Proper use of Jest matchers and assertions (25%)
- Edge cases and error scenarios tested (25%)
- Code organization with describe blocks (10%)
- Descriptive test names following "should..." pattern (10%)

Test Coverage: Beyond the Happy Path

While testing the "happy path" is crucial, real-world applications interact with external systems, databases, and APIs. To truly achieve **comprehensive test coverage** and maintain fast, isolated unit tests, we turn to **mocking dependencies**. Mocking allows you to replace external or complex parts of your system with controlled, simulated versions.

Why Mocking is Essential:

- **Isolation:** Ensures your unit tests only verify the code under test, not its dependencies. If a dependency changes, your unit tests for unrelated components won't break.
- **Speed:** Avoids slow network requests, database operations, or complex calculations. Mocked dependencies respond instantly, making test suites run significantly faster.
- **Control:** Allows you to simulate specific scenarios (e.g., API errors, database failures, specific return values) that might be difficult or impossible to reproduce with real dependencies.

Mocks, Stubs, and Spies:

- **Mock:** A completely fake object that records interactions (e.g., "was this function called?", "what arguments was it called with?") and can have pre-programmed behavior.
- **Stub:** A minimal replacement for a component that provides predefined answers to method calls during a test, without recording interactions.
- **Spy:** Wraps an existing function or method to record its calls and return its original implementation, allowing you to observe behavior without changing it. Jest's mocks can act as stubs and spies.

Jest provides powerful tools for mocking. `jest.fn()` creates a mock function, giving you full control over its return values and allowing you to assert how it was called. `jest.mock('module-name')` is used to automatically mock an entire module, replacing all its exports with mock functions.

Mocking is particularly useful for **asynchronous operations** like API calls or database queries, letting you control resolved or rejected promises.

Consider a `userService.js` module:

```
// userService.js
const axios = require('axios');
const db = require('./database');

async function createUser(userData) {
  // Validate email doesn't exist
  const existing = await db.findUserByEmail(userData.email);
  if (existing) throw new Error('Email already exists');

  // Call external API to verify email
  await axios.post('https://api.emailverify.com/check', {
    email: userData.email
  });

  // Save to database
  return await db.saveUser(userData);
}

module.exports = { createUser };
```

Here's how you'd write Jest tests for it, demonstrating mocking asynchronous dependencies:

```
// userService.test.js
jest.mock('axios');
jest.mock('./database');

const axios = require('axios');
const db = require('./database');
const { createUser } = require('./userService');

describe('createUser', () => {
  beforeEach(() => {
    jest.clearAllMocks();
  });

  test('creates user when email is unique', async () => {
    // Arrange - Mock dependencies
    db.findUserByEmail.mockResolvedValue(null);
    axios.post.mockResolvedValue({ data: { valid: true } });
    db.saveUser.mockResolvedValue({ id: 1, email:
'test@example.com' });

    // Act
    const result = await createUser({ email:
'test@example.com', name: 'John' });

    // Assert
    expect(result.id).toBe(1);
    expect(db.saveUser).toHaveBeenCalledWith({ email:
'test@example.com', name: 'John' });
  });

  test('throws error when email exists', async () => {
    db.findUserByEmail.mockResolvedValue({ id: 2 });

    await expect(createUser({ email: 'existing@example.com' }))
      .rejects.toThrow('Email already exists');
  });
});
```

Pro Tip: For complex interactions, consider using a mocking library like `Nock` (for HTTP requests) or Jest's own `mockImplementationOnce` for specific test cases. Always use `beforeEach(() => jest.clearAllMocks());` to reset mocks between tests, ensuring isolation.

Study Material Exercise

Exercise

Implement an `orderService.processOrder()` function that: 1) calls `inventoryService.checkStock()`, 2) calls `paymentService.charge()`, 3) calls `emailService.sendConfirmation()`, and 4) calls `db.saveOrder()`. Mock all dependencies and write tests covering a success scenario, an out-of-stock scenario, and a payment failure scenario.

Expected Artifact

Complete test file with mocked dependencies, 5+ test cases covering different scenarios, verification that mocked functions are called with correct parameters.

Assessment

Type: Code review + test execution

Criteria:

- All external dependencies properly mocked (35%)
- Verification of mock function calls with correct params (30%)
- Error scenarios tested (20%)
- Proper use of beforeEach for cleanup (10%)
- Tests run in under 100ms total (5%)

WEEK 8 – Authentication & Security

Authentication and authorization form the bedrock of application security. This week transitions from testing fundamentals to securing your applications against real-world threats. You'll implement industry-standard authentication mechanisms, understand role-based access control, and learn to defend against the OWASP Top 10 vulnerabilities. By the end of Week 8, you'll be equipped to build secure, production-ready authentication systems.

SECURITY FOCUS

AUTHENTICATION

CORE CONCEPTS

Authentication vs Authorization: Understanding the Distinction

Authentication

Definition: Authentication answers the fundamental question "Who are you?" It's the process of verifying a user's identity through credentials they provide—typically a username/email and password, but increasingly through biometrics, hardware tokens, or multi-factor methods.

Real-world Analogy: Showing your driver's license to enter a building—proving you are who you claim to be.

Authorization

Definition: Authorization answers "What are you allowed to do?" Once identity is confirmed, authorization determines which resources the authenticated user can access and what actions they can perform. This is typically governed by roles, permissions, and access control policies.

Real-world Analogy: Your employee badge gets you in the building (authentication), but only certain doors unlock for you based on your department (authorization).

Common Authentication Methods in Modern Applications

JWT (JSON Web Tokens): Self-contained tokens that encode user information and claims. The server signs tokens with a secret key, and clients include them in request headers. Stateless and scalable—the server doesn't need to maintain session state. Perfect for microservices and mobile apps. Tokens contain three parts: header (algorithm), payload (claims), and signature (verification).

OAuth 2.0: An authorization framework enabling third-party applications to obtain limited access to user accounts. Users authenticate with a provider (Google, GitHub, Facebook) and grant permissions without sharing passwords. Your app receives an access token to make API calls on the user's behalf. Essential for "Sign in with Google" functionality.

Session-Based Authentication: Traditional server-side approach where the server maintains session state. After successful login, the server creates a session ID stored in a database or memory cache. This ID is sent to the client as a cookie. On subsequent requests, the server validates the session ID and retrieves user data. Stateful but simpler to implement and easier to revoke access.

Security Best Practice: Never store passwords in plain text. Always hash passwords using strong algorithms like bcrypt, Argon2, or scrypt with appropriate salt values. These algorithms are designed to be computationally expensive, making brute-force attacks impractical.

Exercise Activity

Build a login/registration API endpoint using JWT authentication. Implement password hashing, token generation, token validation middleware, and refresh token logic.

Expected Artifact

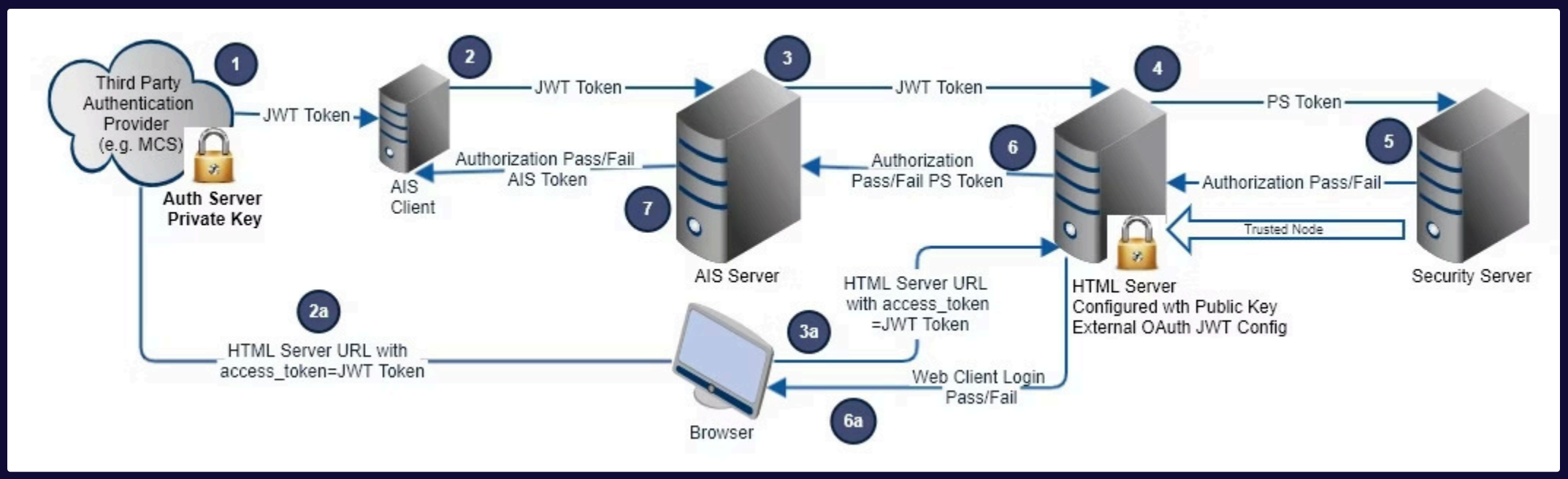
Working API with endpoints for /register, /login, /refresh, /protected. Include Postman collection demonstrating full authentication flow with proper error handling.

Assessment Details

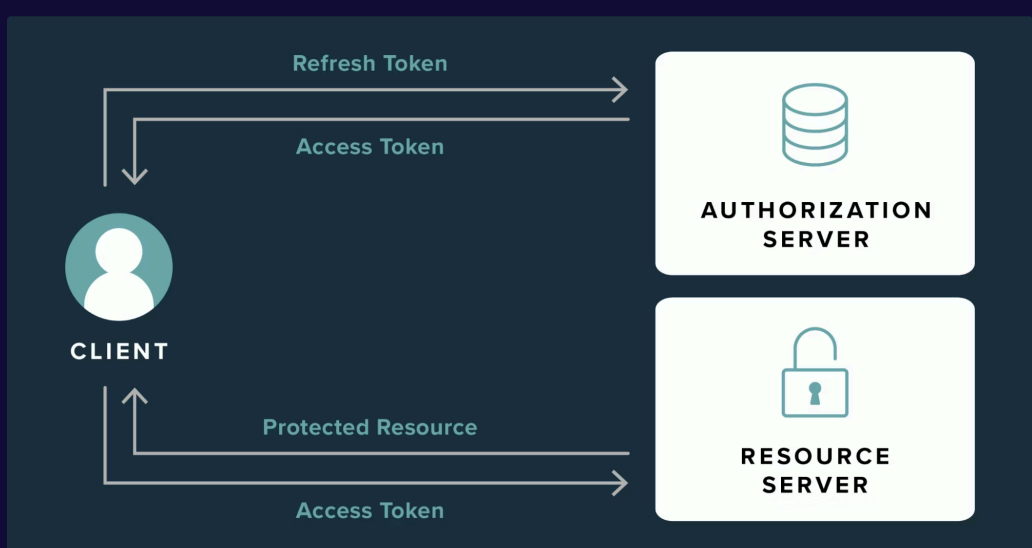
Type: Code review + functional testing

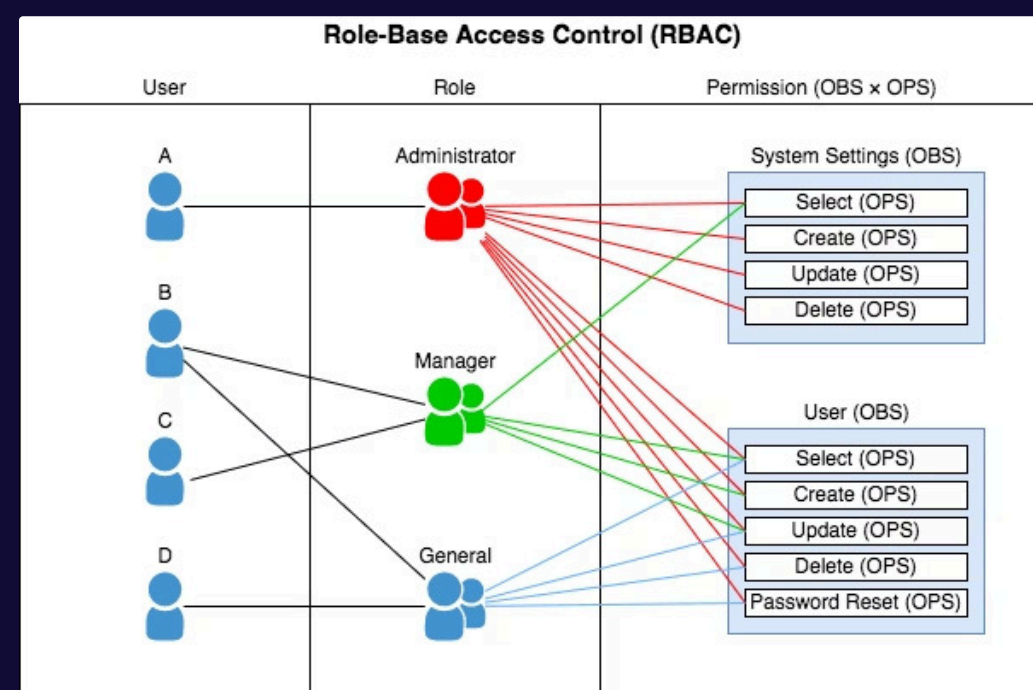
Criteria: Secure password hashing (25%), proper JWT implementation (30%), middleware validation (25%), error handling (20%)

JWT Authentication Flow: Deep Dive



JSON Web Tokens provide a compact, URL-safe method for securely transmitting information between parties as a JSON object. The token's information is verified and trusted because it's digitally signed using either HMAC algorithm with a secret key or RSA/ECDSA public-private key pairs. JWTs are particularly powerful in distributed systems because they eliminate the need for centralized session stores—all necessary information is encoded in the token itself.





JWT Token Structure

A JWT consists of three Base64-encoded parts separated by dots: `header.payload.signature`

01

Header

Contains metadata about the token type (JWT) and signing algorithm (HS256, RS256, etc.). Example: `{"alg": "HS256", "typ": "JWT"}`

02

Payload

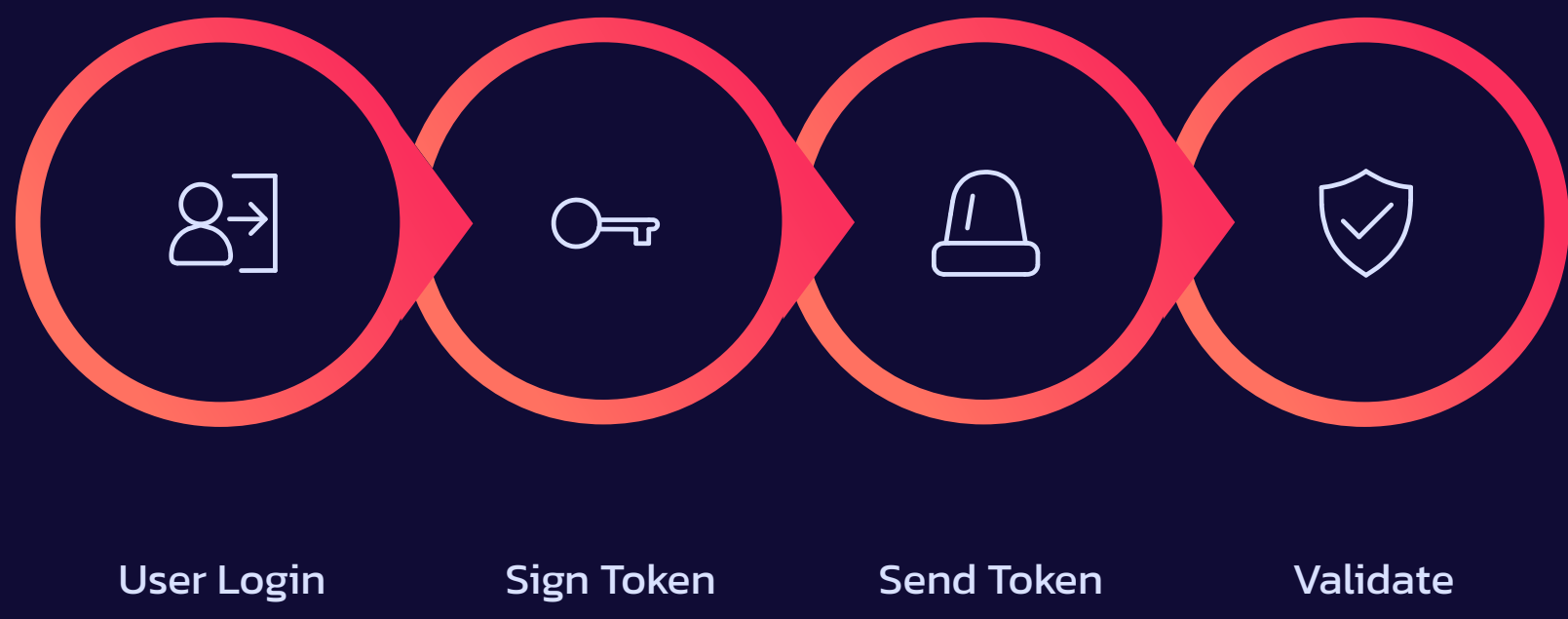
Contains claims—statements about the user and additional metadata. Standard claims include `sub` (subject/user ID), `exp` (expiration timestamp), `iat` (issued at), and custom claims like roles or permissions.

03

Signature

Created by encoding header and payload, then signing with secret key. This prevents tampering—any modification invalidates the signature. Format: `HMACSHA256(base64(header) + "." + base64(payload), secret)`

Complete JWT Authentication Workflow



Understanding the complete authentication flow is critical for implementing secure JWT-based systems. Each step has specific security implications and potential failure points that must be handled correctly.

Step 1: User Login

Client submits credentials (username/email + password) via POST request to /login endpoint. Server validates credentials against stored hashed password using bcrypt.compare(). If invalid, return 401 Unauthorized immediately. If valid, proceed to token generation. **Security consideration:** Implement rate limiting to prevent brute-force attacks—max 5 failed attempts per IP per hour.

Step 3: Send Token in Header

Client stores token securely (HttpOnly cookie or secure storage, never localStorage for sensitive apps). For each authenticated request, client includes token in Authorization header: `Authorization: Bearer eyJhbGciOiJ...`. The Bearer scheme indicates token-based authentication. Server extracts token from header for validation.

1 2 3 4

Step 2: Generate Token

Server creates JWT payload with user identifier, roles, and expiration time (typically 15-60 minutes). Signs token using secret key stored in environment variables (never hardcode secrets). Returns token in response body: `{"token": "eyJhbG...", "expiresIn": 3600}`. Optionally issue refresh token with longer lifespan (7-30 days) for obtaining new access tokens without re-authentication.

Step 4: Middleware Validates

Authentication middleware intercepts request before reaching route handlers. Extracts token, verifies signature using secret key, checks expiration timestamp. If valid, decodes payload and attaches user information to request object for downstream handlers. If invalid/expired, returns 401 Unauthorized. **Critical:** Always verify signature—never trust client-provided data.

Exercise Activity

Implement the complete JWT flow in Node.js/Express or Python/Flask. Include token expiration handling, refresh token logic, and middleware that protects specific routes. Add unit tests for token generation and validation functions.

Assessment Deliverables

Expected Output: Fully functional authentication system with documented API endpoints, error responses, and test coverage >80%.

Assessment Type: Live code demo + security review

Criteria: Working flow (35%), security best practices (30%), error handling (20%), code quality (15%)

RBAC: Role-Based Access Control

Role-Based Access Control (RBAC) is an authorization paradigm where permissions are assigned to roles rather than individual users. Users are then assigned one or more roles, inheriting all associated permissions. This approach dramatically simplifies permission management in large systems—instead of managing permissions for thousands of users individually, you manage a handful of roles.

Core RBAC Concepts

Roles: Named collections of permissions representing job functions (Admin, Manager, User, Guest). Roles should map to real organizational responsibilities. A user can have multiple roles—someone might be both a Manager and a Developer.

Permissions: Granular access rights to specific resources and operations. Format typically follows: `resource:action` (e.g., `users:read`, `users:write`, `posts:delete`). Permissions are atomic—each represents a single capability.

Assignment: The mapping between users and roles. Stored in database tables: `users`, `roles`, `permissions`, `user_roles`, `role_permissions`. Many-to-many relationships allow flexible permission structures.

Design Principle: Follow the *Principle of Least Privilege*—grant users the minimum permissions necessary to perform their job functions. Start restrictive and expand access as needed, never the reverse.

RBAC Implementation Example

Admin Role

Permissions: Full system access including user management, configuration changes, data deletion, analytics access, and audit logs. Can perform any operation on any resource.

Use Case: System administrators, CTO, IT security team.

Risk Level: Highest—compromised admin accounts can destroy entire systems. Require MFA and audit all admin actions.

Manager Role

Permissions: Read/write access to team resources, view team member data, approve requests, generate reports. Cannot modify system settings or access other teams' data.

Use Case: Department heads, team leads, project managers.

Risk Level: Medium—can view sensitive team data but limited system-wide impact.

User Role

Permissions: Read-only access to public resources, edit own profile, submit requests, view own data. Cannot modify other users' data or access admin features.

Use Case: Regular employees, customers, standard application users.

Risk Level: Low—minimal access limits potential damage from compromised accounts.

Exercise

Design and implement an RBAC system with 4 roles (Admin, Editor, Viewer, Guest) and 10+ permissions. Create middleware that checks user roles before allowing access to protected routes. Include a UI for role assignment.

Expected Artifact

Database schema diagram, API endpoints for role/permission management, working middleware demonstrating role checks, and documentation explaining the permission model.

Assessment

Type: Design review + implementation testing

Criteria: Schema design (25%), middleware logic (30%), security (25%), documentation (20%)

OWASP Top 10: Security Vulnerabilities

The Open Web Application Security Project (OWASP) maintains a list of the most critical web application security risks. Understanding these vulnerabilities is essential for building secure applications. These aren't theoretical threats—they're actively exploited in the wild, resulting in billions of dollars in damages annually.

1

SQL Injection

Attack: Malicious SQL code inserted through user input fields, allowing attackers to read, modify, or delete database contents.

Example: Input: `admin' OR '1'='1` bypasses authentication.

Defense: Always use parameterized queries/prepared statements. Never concatenate user input into SQL strings. Use ORM frameworks that handle escaping automatically.

2

Cross-Site Scripting (XSS)

Attack: Injecting malicious JavaScript into web pages viewed by other users. Attackers steal cookies, session tokens, or perform actions as the victim.

Example: Comment field: `<script>alert(document.cookie)</script>`

Defense: Sanitize all user input before displaying. Use Content Security Policy headers. Encode output based on context (HTML, JavaScript, URL).

3

Broken Authentication

Attack: Exploiting weak authentication mechanisms—credential stuffing, session hijacking, predictable session IDs, missing MFA.

Example: Weak password requirements allowing "password123" or lack of account lockout after failed attempts.

Defense: Enforce strong password policies, implement MFA, use secure session management, set proper session timeouts, and monitor for suspicious login patterns.

4

Sensitive Data Exposure

Attack: Accessing sensitive data transmitted or stored without proper encryption—credit cards, health records, credentials sent over HTTP.

Example: Database backup with unencrypted passwords, API responses leaking Social Security numbers.

Defense: Encrypt data at rest and in transit (TLS/HTTPS). Use strong encryption algorithms (AES-256). Never log sensitive data. Implement proper key management.

Comprehensive Security Learning Path

Detailed Study Materials

- **OWASP Documentation:** Read complete OWASP Top 10 guide with examples and mitigation strategies
- **Penetration Testing:** Learn to use Burp Suite, OWASP ZAP for vulnerability scanning
- **Secure Coding:** Review language-specific security guides (OWASP Cheat Sheets)
- **Cryptography Basics:** Understand hashing, encryption, signing, and when to use each
- **Security Headers:** Study CSP, HSTS, X-Frame-Options, X-XSS-Protection

Practical Lab Exercises

1. Exploit intentionally vulnerable apps (DVWA, WebGoat, Juice Shop)
2. Fix security vulnerabilities in sample codebases
3. Implement security headers and test with Observatory by Mozilla
4. Conduct threat modeling session for a sample application
5. Perform security code review on peer projects

Final Capstone Project

Exercise: Build a complete secure web application incorporating all learned concepts: JWT authentication, RBAC authorization, protected against OWASP Top 10.

Requirements:

- User registration/login with JWT
- 3+ roles with different permissions
- Input validation preventing injection attacks
- XSS protection throughout
- HTTPS enforced
- Security headers configured
- Comprehensive test suite

Assessment Type: Security penetration test + code review

Criteria:

- Authentication implementation (20%)
- Authorization logic (20%)
- OWASP vulnerability prevention (30%)
- Test coverage (15%)
- Code quality and documentation (15%)

Passing Score: 80% with no critical security flaws